

CSCI 3081W - Program Design and Development

Homework 2 - UML Basics and Design

Spring 2025

This document is subject to minor changes regarding phrasing, grammar, spelling, and clarity until Tuesday, February 25th at 11:59 pm.

Due Dates

- **Full Paper** - Friday, March 7th 2024, before 11:59 pm.

Overview

This homework consists of five questions. The first question will focus on learning UML Class diagram concepts and answering questions in Gradescope based on provided explanations and examples. In questions 2 – 5, the instructions will vary and each question will have a UML Class diagram component as well as a writing component.

Make sure to start early! Design and documentation is a critical and time-intensive part of software development. You won't be able to complete this assignment if you start it the day that it's due.

This is an individual assignment. Looking at someone else's UML Class diagram(s) is considered academic dishonesty.

We will be using [Mermaid Class Diagram](#) syntax for this assignment. You can use [Mermaid's in browser editor](#) to generate UML Class diagrams. It is your responsibility to thoroughly read the documentation before starting this assignment.

Reminder: this is a writing intensive course. The writing standards for this assignment were discussed in lecture and are also here for your convenience. We expect correct grammar and spelling as well as keeping your writing concise, clear, and complete.

Problem 1: (20 points)

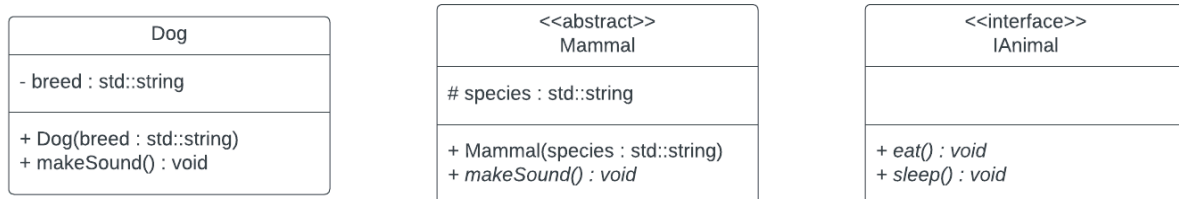
After reading the following UML class diagram guide, you will take a quiz on Gradescope which is linked at the bottom of this section.

UML, or Unified Modeling Language, is a standardized modeling language used in software engineering. It primarily consists of a set of well-defined and documented notations and conventions for creating graphical representations of software systems. When used correctly, UML is a powerful and widely-understood visual language for both modeling and documenting object-oriented design.

In UML, there are a variety of diagrams that can be employed. The most widely-used is the Class Diagram, depicting a system design in a (typically) object-oriented viewpoint. Various components are used to represent different aspects of a system's structure and relationships: classes, attributes, methods, relationships, and notes. The next section will define each of these components individually and provide an example usage for each of them.

Classes

Classes represent a blueprint for objects in a system. Classes include attributes and methods. There are special types of classes called abstract classes and interfaces. Abstract classes are classes that can't be instantiated and usually serve as a base class for other classes. Interfaces represent a set of operations that a class can implement (which means they also cannot be instantiated). In both Abstract and Interface cases, we use the UML Stereotype, indicated by the double angle brackets (i.e., "<<" and ">>"), to signal those roles within our diagrams.



In the image above, you can see three different examples of classes that can appear in a UML diagram, each with its own name, attributes, and methods. Each class diagram box is split horizontally so there are three¹ separate sections. The top section holds the class name. The other two sections will be discussed further down.

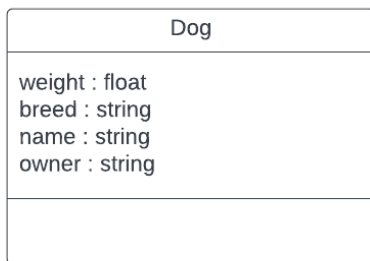
The left class in the image above, `Dog`, is a regular class. `Mammal`, in the middle, is an abstract class, indicated by adding the `<<abstract>>` stereotype above the class name. Abstract

¹ Technically, you can have more than three sections, but we have never seen a real-world instance of such a diagram. Every instance we've seen has three or fewer sections, depending on the stage of modelling. See "Unified Modeling Language Reference Manual" p. 188 for more information if you're interested, but we won't be using any more than the standard three sections going forward.

classes in C++ must contain at least one *pure* virtual function. Recall what we practiced in workshop 2 – a virtual function is one where subclasses can override the method or use the existing definition in the base class. A pure virtual function is declared in the base class by italicizing the function name, which means any subclasses **MUST** provide their own definition. In this example, notice that the *makeSound()* function is italicized. This is the UML notation that indicates that it is an abstract operation. The last class in our example set, *IAntimal*, is an interface which we indicate by starting the name with an “I” and adding the stereotype <<interface>> above the class name. Interfaces **only** contain pure virtual functions. Notice that *eat()* and *sleep()* are both italicized, indicating that they are pure virtual functions.

Attributes

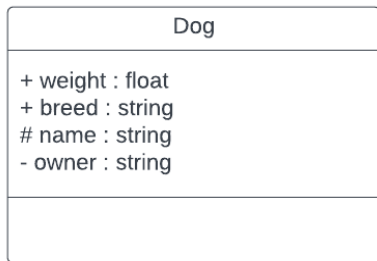
Attributes are the characteristics or properties of a class. These are also known as member variables or data members. Attributes are distinct and independent of each other; they may hold a value or be a placeholder for a value of the class object. They are listed within the second third of the class diagram box.



On the left, you see a Dog class with four attributes listed in the middle section of the class diagram box. Each attribute has a name and a type, separated by a colon. In this example, we have attributes named *weight*, *breed*, *name*, and *owner*. While *weight* is indicated as a float, the rest are strings.

In UML, the access of attributes can be described by using the following symbols: +, -, and #.

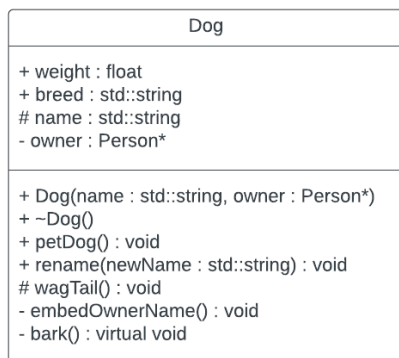
Access	Public +	Protected #	Private -
Same Class	yes	yes	yes
Derived Class	yes	yes	no
Outside Class	yes	no	no



Here to the left you can see that we've now added access modifiers to each of the attributes in the `Dog` class. We have decided that `weight` and `breed` are public. All subclasses of `Dog` should be able to use the `name` attribute, but caution is warranted because it can contain identifying information. Therefore, we've made it protected rather than private. The `owner` attribute contains the most sensitive information and therefore is private, as we only want each class instance to be able to access that information.

Methods

Methods (also known as member functions or behaviors) are functions or operations that the class can perform. They are listed in the final third of the class box. Each function also has an access modifier similar to that of the attributes.



If you look to the left again, you can see that we've now added some methods to the bottom section of the `Dog` class diagram box. The first method defined is a constructor that takes in a string for the dog's name and a pointer to a `Person` (UML Class box for `Person` not shown here) called `owner`. You indicate the name and type of each parameter similarly to the attributes, with the variable name and type separated by a colon.

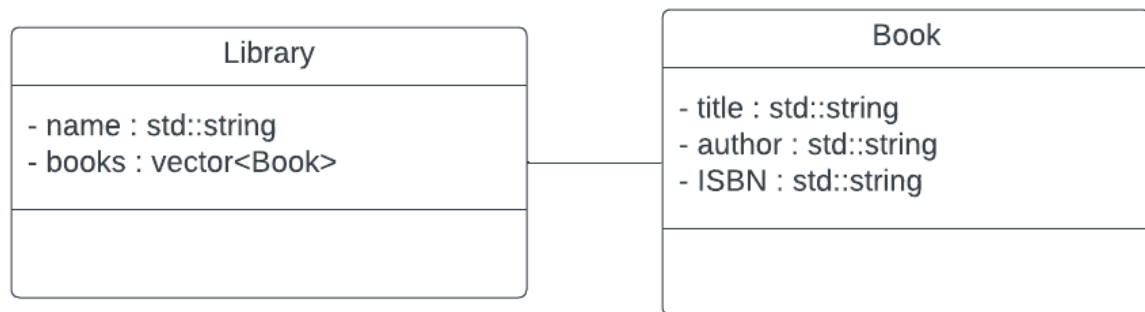
After the constructor we list a destructor, `~Dog()`. In UML, we usually do not declare a return type for constructors or destructors. We also have two public methods called `petDog` and `rename`, both of which are void functions (they return nothing). The `rename` function takes in a string argument representing a new name for the `Dog` object. Unlike what you're used to in C++, in UML you indicate the method's return type *after* the method signature, and the two are again separated by a colon. We then have one protected method called `wagTail` which returns nothing. We make this method protected because we want any future dog subclasses to be able to wag their tails, but it's not something another piece of code can directly control as they could if the method were public (i.e., they can pet the dog, which might cause the dog to decide to wag its tail, but they can't directly make a tail start wagging). We have one private method, which is a helper function called `embedOwnerName` and allows the class to add security to the owner's name. Finally, we have a private method called `bark`. The return type of `bark` is `virtual void` which indicates that the function is a virtual function. Virtual functions can be shown in the return type by starting with `virtual` followed by the return type.

Relationships

Relationships describe how classes connect to each other and how information flows between them. They are represented by different types of arrows between elements. We will cover the following types of relationships: association, dependency, inheritance/generalization, realization, aggregation, and composition.

Association

The association relationship is used to model that a general connection is maintained over time between two² classes. It can signify a unidirectional or a bidirectional relationship where instances of one class are associated with instances of another class. Let's consider two classes: `Library` and `Book`. The `Library` class is associated with the `Book` class because it contains books. The association implies that instances of the `Library` class can have a collection of books, and instances of the `Book` class can be associated with a particular library.



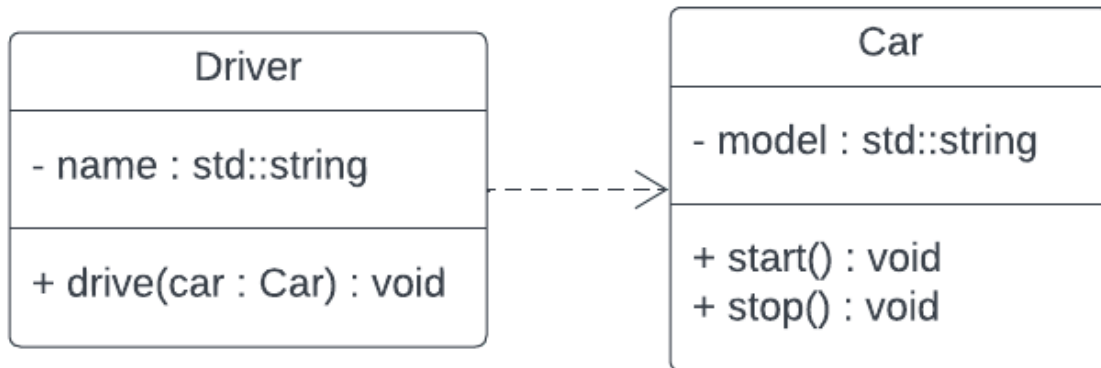
Instances of the `Library` class can be associated with multiple instances of the `Book` class through the `books` attribute, and instances of the `Book` class could be associated with a specific library if needed. The association relationship in this case is a general undirected connection between two classes, indicating that they are somehow related in the model. Undirected associations are common in the early stages of modeling (Analysis & Design) and are nearly all converted to directed associations (or other forms of relationships) in later stages (Implementation & Deployment). We will mostly see directed associations in this class, but may include some undirected associations while doing our design work.

Dependency

The dependency relationship is used to model a relationship between two classes where a change in one class may affect the other class. It is a more general relationship, indicating a transient connection between the two. Let's consider two classes: `Driver` and `Car`. The `Driver` class has a dependency on the `Car` class whenever the driver uses a car, as would be

² Technically, you can have associations between more than two classes. In this class, we usually operate at the Implementation stage of modeling, where we will in nearly all cases be looking at just two classes per Association.

the case when a `Car` object is passed as an argument to the `drive()` method. The dependency relationship implies that changes in the `Car` class – that is, any modifications to its interface or behavior – may impact the `Driver` class.

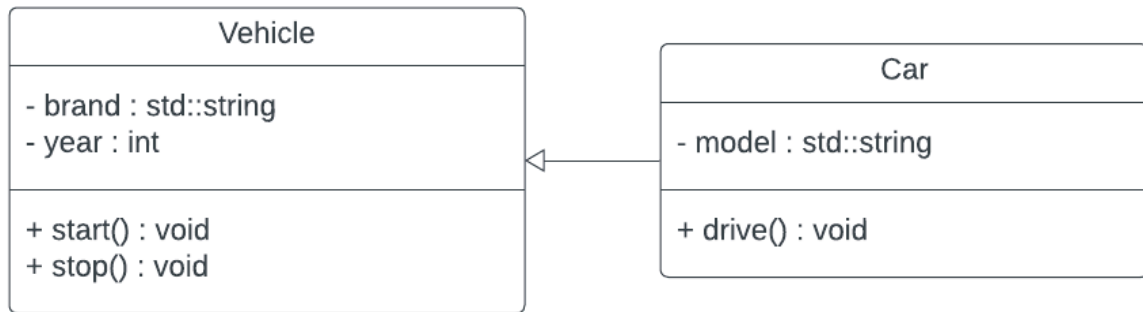


The `Driver` class has a method `drive(car : Car)` which indicates that a driver can drive a car. The `Driver` class depends on the `Car` class through the drive method. The `Driver` class uses the `Car` class and changes to the `Car` class may affect the `Driver` class if it relies on specific public behaviors or attributes of the `Car`. The dependency relationship is indicated by the method parameter and the use of `Car` within the drive method. In UML, we also use visual notation to indicate a dependency relationship. A dashed line with a pointer line ending is how we denote a dependence relationship. Notice that the arrow is going away from the class with the dependency (`Driver`), and towards the class it depends on (`Car`). This direction is very important!

Inheritance/Generalization

The inheritance relationship is used to model the “is-a” relationship between classes. It signifies that one class (the subclass, derived, or child class) inherits attributes and behaviors from another class (the superclass, base, or parent class). Let’s consider two classes: `Vehicle` as the superclass and `Car` as the subclass. The `Car` class inherits attributes and behaviors from

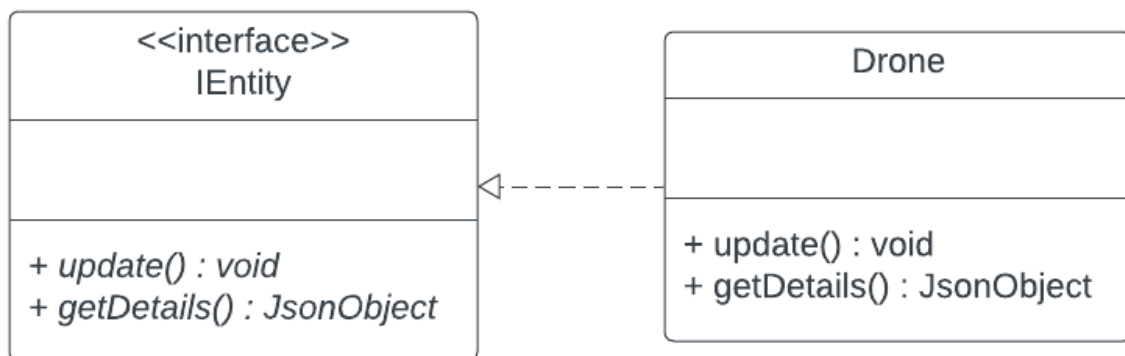
the `Vehicle` class, indicating that a car is a type of vehicle.



The `Vehicle` class is the superclass with attributes `brand` and `year`, as well as two methods called `start()` and `stop()`. The `Car` class is the subclass that inherits from `Vehicle`. It has additional attributes specific to cars (`model`) and additional methods (`drive`). The `Car` class still has a `brand` and `year` member variable and `start()` and `stop()` member functions. Pay close attention to the visual language in our `Vehicle/Car` diagram as well. Inheritance relationships are denoted with a solid line and a white (or blank/not-filled-in) arrowhead. The arrow is always pointing away from the child/subclass and towards the parent/superclass.

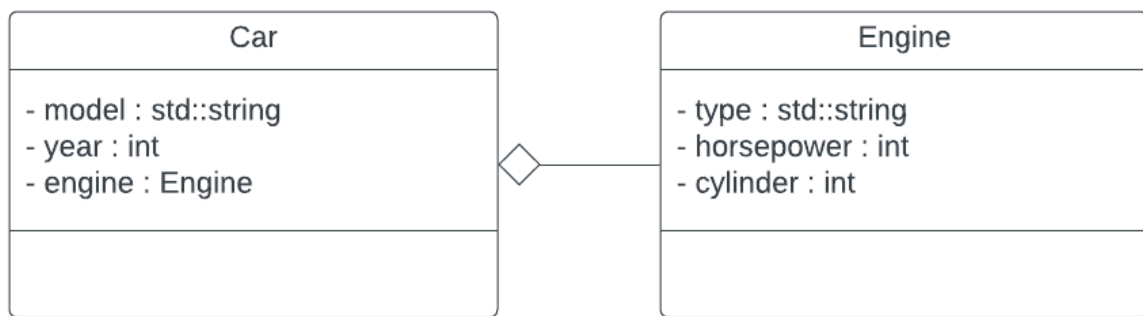
Realization

The realization relationship is used to model the implementation of an interface by a class. Remember that by definition an interface contains only function declarations, never definitions, and can never be instantiated. An interface is only accessible once a class has been created that *realizes*, or makes real, the implementation for the methods defined in the interface. Let's consider two classes: `Drone` and an interface `IEntity`. The `Drone` class realizes the `IEntity` interface, indicating that it provides the actual implementation for the methods defined in `IEntity`. In UML, this relationship is shown using a dashed line and a blank/white arrowhead. The arrow goes away from the concrete class and towards the interface.



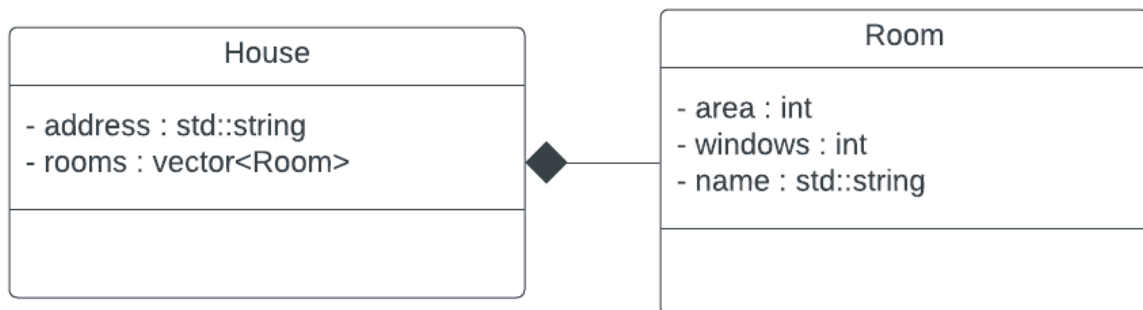
Aggregation

Aggregation is a type of association indicating a “whole-part” relationship between two classes. Let’s consider two classes: `Car` and `Engine`. The aggregation relationship says that a `Car` contains an `Engine`, but the `Engine` can exist independently of a specific car. For example: a car has a specific engine (instance of `Engine` class), and that engine is part of the car, but the engine itself could also exist outside of any specific car. Here, you can see a solid line with a white diamond used to signify the aggregation relationship between `Car` and `Engine`. The diamond is attached to the class that contains instances of the other class; in other words, the line goes from the “part” to and the diamond signifies the “whole”.



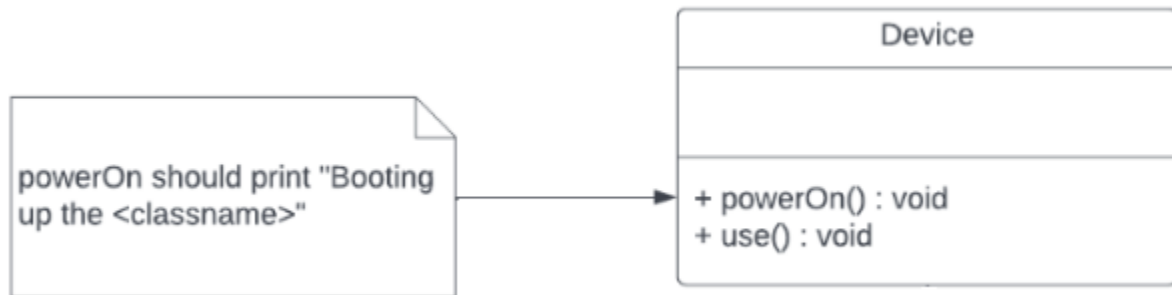
Composition/Composite Aggregation

Composite aggregation is a strong form of aggregation that requires a part object be included in at most one composite object at a time. If a composite object is destroyed, it must either destroy all its parts or else give responsibility for them to other objects. Let’s consider two classes: `House` and `Room`. Just as in the real world, a `House` is composed of `Room` objects, and the `Room` objects are part of the `House`. If the `House` is destroyed, all the `Room` objects contained in `rooms` cease to exist. This reflects a composition relationship. In UML, you use a solid line and a filled-in black diamond to represent this relationship, where the diamond is attached to the class that contains instances of another class.



UML Comments/Notes

Notes in UML are used to comment on or explain a specific part of the diagram. These notes are attached to relevant parts of the diagram. These are particularly useful for including helpful information that doesn't fit well using the existing syntax of UML.



In this case, the UML note describes how the `powerOn` method operates. UML notes can also describe other aspects of the class such as member variables or relationships to other classes.

Cardinality and Multiplicity

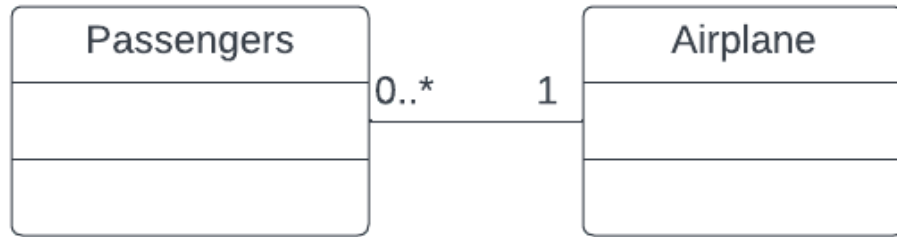
Cardinality is the number of elements in a set. It is a specific number. This is in contrast to multiplicity, which is the range of possible cardinalities a set may hold.

Multiplicity is a specification of the range of allowable cardinality values - the sizes - that a collection may assume. Multiplicity specifications may be given for association ends, attributes, or parts within composite classes as well as other purposes. In principle, a multiplicity is a (possibly infinite) subset of the non-negative integers. In practice, it is an integer interval. If multiplicity is greater than one, it includes an indication of whether the elements are ordered and unique.

Multiplicities can be used to model the relationship between two classes. For example, an `Airplane` class can have an attribute `vector<Passenger>` which is a list of the passengers on board. Since the amount of passengers is between zero and some positive integer, we can model this relationship in UML with a 1 to 0..* UML relationship.

Examples:

0..1	optional value
1	exactly one
0..*	any number of elements, unordered, unique
1..3	between 1 and 3



Gradescope Quiz

The Gradescope quiz is titled Homework 2 - Problem 1.

Problem 2: (20 points)

Instructions:

Below is an AI-generated UML Diagram for an **Online Store System**. However, the AI has made several mistakes in relationships, missing attributes/functions, and incorrect encapsulation.

Flawed AI-Generated UML Diagram:

```
classDiagram
    class Customer {
        +String name
        +String email
        +int customerID
    }
    class Order {
        +Date date
        +int orderID
    }
    class Product {
        +String name
        +double price
    }
    Customer "*" -- "*" Order : places
    Order "*" -- "*" Product : contains
```

Problem Description: Online Store System

Given the flawed AI-generated UML Diagram and the following requirements, find mistakes the AI has made, correct them, and provide a short explanation to justify your corrections. The Online Store System should meet the following requirements:

1. Customer Class:
 - a. Attributes: name (String, public), email (String, public), customerID(int, private)
 - b. Methods: getCustomerID() to retrieve the customer ID.
 - c. A customer can place **multiple orders** (one-to-many relationships with **Order**)
2. Order Class:
 - a. Attributes: date (Date, public), orderID (int, private)

- b. Methods: getOrderID() to retrieve the order ID.
 - c. An order can contain **multiple products** (many-to-many relationships with **Product**).
- 3. Product Class:
 - a. Attributes: name (String, public), price (double, private)
 - b. Methods: getPrice() to retrieve the price.

Problem Requirements:

Part 1: Identify mistakes

- There are multiple mistakes in the AI-generated UML Diagram. Please identify at least 4 mistakes in the UML Diagram.

Part 2: Correct the UML Diagram

- After you find the mistakes in the UML Diagram, please correct them using Mermaid syntax (the same syntax as the given UML Diagram).

Part 3: Justify your corrections

- After you find the mistakes and correct them, please provide an explanation to justify your corrections.

Submission Requirements:

- A corrected UML class diagram using Mermaid syntax.
- An explanation justifying your corrections.

Problem 3: (20 points)

Instructions:

GPT, with some guidance, was tasked with designing a class-based structure for a lightweight CRPG battle simulator with UML. Your goal: spot **5 mistakes** in GPT's solution, provide your own corrections, and explain them.

Problem Description:

System Design Outline:

Character (Base Class)

This is the base class for all characters (both players and monsters).

It includes the following member variables:

name (string)

health (int)

attack (int)

defense (int)

speed (int)

team (Team *)

And these member functions:

Character(const string & name, int health, int attack, int defense, int speed, Team *team);

void takeDamage(int damage);

virtual void performAction(Character *target) = 0; // pure virtual

Player (Derived from Character)

The Player class represents a controllable character.

It has additional variables:

level (int)

inventory (vector<Item *>)

And member functions:

Player(const string & name, int health, int attack, int defense, int speed, Team *team, int level, const vector<Item *> & inventory);

void performAction(Character *target);

void useItem(Item *item, Character *target);

Monster (Derived from Character)

The Monster class represents enemy combatants and contains:

loot (vector<Item *>)

And member functions:

```
Monster(const string & name, int health, int attack, int defense, int speed, Team *team, const  
vector<Item *> & loot);  
void performAction(Character *target);
```

Item

The Item class represents consumable or lootable objects.

It has the following member variables:

```
name (string)  
effectValue (int)  
targetType (int)
```

And member functions:

```
Item(const string & name, int effectValue, int targetType);  
void applyEffect(Character *target);
```

Battle

The Battle class manages combat between two teams:

```
team1 (Team *)  
team2 (Team *)
```

Functions:

```
Battle(Team *team1, Team *team2);  
void startBattle();  
bool isBattleOver();  
Character *getFastestCharacter();
```

Team

The Team class represents a collection of characters in a party.

Member variables:

```
members (vector<Character *>)  
teamName (string)
```

Functions:

```
Team(const vector<Character *> & members, const string & teamName);  
void addMember(Character *character);  
bool isDefeated();
```

GameManager

This class manages game flow, character creation, and battle sequences.

Member variables:

```
players (vector<Player *>)  
monsters (vector<Monster *>)  
itemPool (vector<Item *>)
```

battles (vector<Battle *>)

Functions:

```
GameManager(const vector<Player *> & players, const vector<Monster *> & monsters, const  
vector<Item *> & itemPool, const vector<Battle *> & battles);  
void initializeGame();  
void startBattle(Team *playerTeam, Team *monsterTeam);
```

GPT's solution in mermaid code:

See hw2_p3_gpt.txt

Additional Problem Requirements:

- You should try to find mistakes in GPT's solution, not in the system design outline.
- You don't need to consider getter/setter functions, or any other member variable / function that is not shown in the outline even if you believe it's logically necessary.
- Neither cardinality nor multiplicity is required for both GPT and you, so it should never be considered a mistake.
- Mermaid uses labels for describing relationships. However, the labels should not be considered mistakes even if they don't really make sense.
- Anything only syntactically different from the guide in problem 1 is not a mistake. E.g. the pure virtual function is not italicized in mermaid.
- There are more than 5 mistakes included, but you only need to find 5. If you submit more than 5, then only the first 5 are going to be graded.
- Only the following are considered mistakes:
 - Something is missing (omissions)
 - A relationship needs to be changed (i.e. you remove one relationship and replace it with another. This still counts as only one mistake).

Submission Requirements:

- For each of the 5 mistakes you need to include 3 items in your answer:
 - What GPT did wrong in its solution
 - Your correction
 - Rationale behind your change(s)

Problem 4: (20 points)

Instructions:

In this problem, you will leverage AI-generated outputs to critically analyze and improve a UML diagram design. You are required to work with outputs generated from two different AI models. Your tasks include comparing the diagrams, discussing their strengths and weaknesses, and synthesizing a final, integrated diagram that incorporates the best elements from each and the UML principles taught in the lecture.

Problem Description:

Part 1: AI Generated UML Class Diagrams

As a member of a Computer Science Club, you have been requested to create a **hackathon management system** that allows students to register, form teams, submit projects, and vote for winners. Since you have a small team, you outsource some planning work to **Generative AI**. So, you give both **ChatGPT** and a **different generative AI tool** the prompt below to create the UML diagram for the system:

"Create a mermaid UML class diagram for a hackathon management system. The system should allow students to register, form teams, submit projects, and vote for winners. Each student has a name, email, and a list of hackathons they've joined. Teams should have a name, a list of members, and a project submission. Projects should have a title, description, GitHub link, and votes from other students. The system should also track judges who can review and score projects."

Part 2: Analysis and Reflection

Write a clear, concise analysis addressing the following questions:

Properties included in each class

- Did the models correctly implement all properties such as team name, project GitHub link, votes)?
- Were some properties over-simplified or missing?

Class structure and abstractions

- Did AI create a separate Team class to handle members, or did it just store them in the Student class?
- Did AI recognize the need for a Registration class to link students to hackathons?

Data types and relationships

- Were data types appropriate for attributes like votes and scores?
- Were the relationships between entities well captured?

Methods and interactions

- Did AI include methods for forming teams, submitting projects, and voting?
- Did it account for judges scoring projects separately from student votes?

Feedback loops and missing interactions

- Did AI include notifications or leaderboard updates?
- Were there features to prevent duplicate votes or track project revisions?

Compare and Contrast

- Write out your discussion of which properties, methods, relationships, or additions are the same or different between the two results or outputs. Try to reason why you see the results that you do.

Part 3: Combining the Diagrams

Create a UML diagram that integrates the strengths from both generated results.

Submission Requirements:

It is your responsibility to make sure that your revised UML diagram is as complete and correct as possible.

- 2 screenshots from ChatGPT and the other generative AI tool of your conversation histories
- 1 written discussion of the comparison of two outputs
- 1 revised UML diagram that integrates the strengths of both results

Problem 5: (20 points)

Instructions:

You will design a UML Class diagram for a software system manually and then generate an AI-created UML diagram using a tool like chatGPT or DeepSeek. You will analyze the differences, evaluate the design choices made by both yourself and the AI, and explore potential improvements.

You must cite which generative AI tool you are using as well as the chat history with that model. For example, "I used chatGPT o4 and the history of our chat is pasted below: ".

Problem Description: Advanced Library Management System

Design a Library Management System that supports multiple types of users and advanced functionality. Your system should include the following requirements:

1. User Roles and Permissions
 - a. The system must support the following users: `Library Members`, `Librarians`, and `Administrators`.
 - b. `Library Members` can search for, borrow, and return books.
 - c. `Librarians` can add and remove books, manage inventory, and approve special book requests.
 - d. `Administrators` can manage users, modify system policies (such as loan durations and late fees), and generate reports.
2. Books and Inventory Management
 - a. A `Book` should have attributes including title, author, ISBN, genre, publication year, and availability status.
 - b. A `Book` may have multiple copies, and the system must track the number of available copies.
 - c. A `Book` can be categorized into `PhysicalBooks` and `E-books`.
3. Borrowing and Reservations
 - a. A `Book Borrowing System` should track due dates, overdue books, and automatic late fee calculation.
 - b. Users should be able to reserve books if they are unavailable. The system should notify them when a book becomes available.
 - c. The system must enforce borrowing rules, such as limiting the number of books a user can borrow at a time.
4. Search and Recommendations
 - a. The system should allow users to search for books by title, author, genre, and availability.

- b. The system should provide book recommendations based on a user's borrowing history.
- 5. System Reports and Logging
 - a. The system should generate reports on most borrowed books, overdue books, and user activity.
 - b. The system should maintain logs of all borrowing and returning activities.

Problem Requirements:

Part 1: Manual UML Class Diagram

Create a detailed UML class diagram that includes:

- At least six classes with appropriate names.
- Attributes and methods for each class.
- Proper relationships (inheritance, composition, or aggregation).
- Appropriate visibility modifiers (private, public, protected).
- Associations with multiplicity (e.g., one-to-many, many-to-many).

Part 2: AI-Generated UML Diagram

- Use an AI tool (e.g., ChatGPT) to generate a UML class diagram based on the same system description.
- Record the prompt you used and any modifications you made to improve the AI's output.

Part 3: Analysis & Reflection

Write a clear, concise analysis addressing the following questions:

Comparison & Evaluation

- How do the number of classes and relationships compare between your diagram and the AI-generated diagram?
- Did the AI capture all required details, or were any components missing or incorrect?
- Were there unexpected design choices in the AI's diagram? If so, what were they?

Design Choices & Engineering Considerations

- How did you decide on class responsibilities and relationships?
- If the AI introduced a different structure, do you think it improves the design? Why or why not?
- Did you follow any software design principles (e.g., SOLID)? If so, which ones and why?

Iterating on AI's Output

- If you modified the AI's output, what changes did you make, and why?
- If you modified your diagram, what changes did you make, and why?
- Could you engineer better prompts to improve the AI's UML diagram? If so, how?

Broader AI & Software Engineering Questions

- How could AI-assisted UML generation be useful in software development?
- What limitations did you notice in AI's ability to generate UML diagrams?

- Do you think AI can replace human engineers in software design? Why or why not?

Part 4: Iterating on the Manual UML Class Diagram

Edit your first UML class diagram with any improvements from your generative AI tool.

Submission Requirements:

It is your responsibility to make sure that your revised UML diagram is as complete and correct as possible.

- First iteration of manually created UML class diagram (mermaid code and screenshot of UML class diagram).
- AI-generated UML class diagram (mermaid code, prompt(s) and generative AI model used, and screenshot of UML class diagram).
- Final written reflection answering the analysis questions.
- Second iteration of manually created UML class diagram (mermaid code and screenshot of UML class diagram).